

DENNIS GUNAWAN
DAVID AGUSTRIAWAN



IF351 DATABASE SYSTEMS

10 STORED PROCEDURES AND ADVANCED SQL QUERY (CTE)



OUTLINE

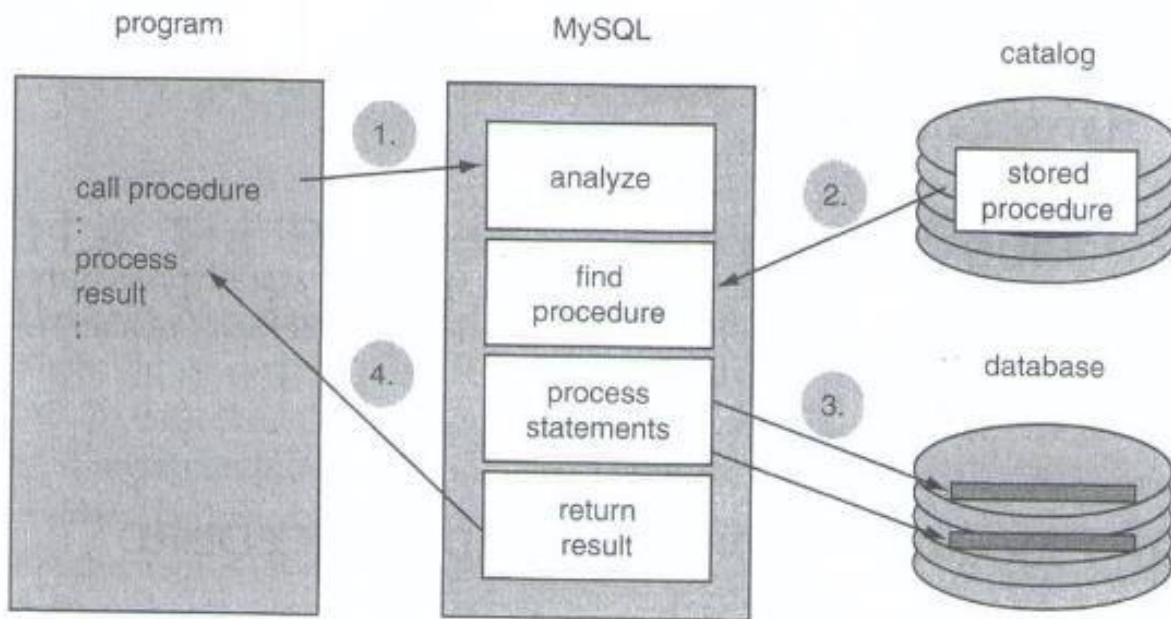
- Stored Procedures
- Local Variables
- Flow-Control Statements
- Querying Data with SELECT INTO
- MySQL Delimiter
- CTEs
- SQL Window Function

STORED PROCEDURES

A certain piece of code (the procedure) consisting of declarative and procedural SQL statements
stored in the catalog of a database
that can be activated by calling it
from a program, a trigger, or another stored procedure

- CREATE PROCEDURE statement consists of several other SQL statements
- Each stored procedure consists of at least 3 parts
 - A name
 - A list of parameters
 - A body

THE PROCESSING STEPS OF A STORED PROCEDURE



1. The process begins when the procedure is called from the program
 2. The database server receives this call and finds the matching procedure in the catalog
 3. The procedure is executed
 4. When the procedure is finished, the result of the procedure is returned
- No communication takes place between the database server and the program during the execution of the procedure

THE PARAMETERS OF A STORED PROCEDURE

- A stored procedure has zero, one, or multiple parameters
- Through these parameters, the procedure is capable of communicating with the outside world
- 3 types of parameters
 - With **input parameters**, data can be passed to a stored procedure
 - The stored procedure uses **output parameters** when an answer or result must be returned
 - **Input/output parameter** can act as an input as well as an output parameter

THE PARAMETERS OF A STORED PROCEDURE

```
CREATE PROCEDURE procedure_name
    ( [ [ IN | OUT | INOUT ] parameter_name data_type,
      [ IN | OUT | INOUT ] parameter_name data_type,
      ... ] )
routine_body
```

- `CREATE PROCEDURE delete_actor ( IN actor_id CHAR(5) )`
...
- `CREATE PROCEDURE show_all_movies ()`
...
 - A stored procedure does not need parameters, but opening and closing brackets are still required

THE BODY OF A STORED PROCEDURE

- The body of a stored procedure contains all the statements that must be executed when the procedure is called
- The body always begins with **BEGIN** and ends with **END**
- In between, all statement types can be specified
 - SQL statements (DDL, DCL, and DML statements)
 - Procedural statements, such as IF-THEN-ELSE and WHILE-DO

THE BODY OF A STORED PROCEDURE

- **CREATE PROCEDURE** delete_actor (IN actor_id CHAR(5))

BEGIN

DELETE

FROM actors

WHERE id = actor_id;

END

- **CREATE PROCEDURE** show_all_movies ()

BEGIN

SELECT id, name, year, rank

FROM movies;

END

```
CREATE PROCEDURE procedure_name
```

```
( [ [ IN | OUT | INOUT ] parameter_name data_type,  
  [ IN | OUT | INOUT ] parameter_name data_type,  
  ... ] )
```

```
BEGIN
```

```
body_statement
```

```
END
```


LOCAL VARIABLES

- Within a stored procedure, local variables can be declared
- They can be used to store temporary intermediate results
- If we need a local variable within a stored procedure, we must introduce it first with a DECLARE statement
- With a declaration, the data type of the variable is determined, and an initial value can be specified

LOCAL VARIABLES

```
DECLARE variable_name, variable_name, ... data_type  
[ DEFAULT default_value ];
```

- DECLARE score DECIMAL(5,2);
- DECLARE name VARCHAR(40);
- DECLARE number1, number2 INTEGER;
- DECLARE number1 INTEGER DEFAULT 100;

THE SET STATEMENT

- SET statement can assign a value to a local variable

```
SET variable_name { = | := } scalar_expression,  
  variable_name { = | := } scalar_expression,  
  ...;
```

- SET var1 = 1;
- SET var1 := 1;
- SET var1 = 1, var2 = var1;

FLOW-CONTROL STATEMENTS:

IF STATEMENT

```
CREATE PROCEDURE odd_or_even ( IN number INTEGER )
BEGIN
    DECLARE output VARCHAR(255);
    SET output = '';
    IF number % 2 = 0 THEN
        SET output = CONCAT(number, ' ', 'is an even number');
    ELSE
        SET output = CONCAT(number, ' ', 'is an odd number');
    END IF;
    SELECT output "Odd Or Even?";
END
```

```
IF condition THEN statement_list
    [ ELSEIF condition THEN statement_list ]
    [ ELSEIF condition THEN statement_list ]
    ...
    [ ELSE statement_list ]
END IF;
```

FLOW-CONTROL STATEMENTS:

CASE STATEMENT

```
CREATE PROCEDURE odd_or_even ( IN number INTEGER )
BEGIN
    DECLARE output VARCHAR(255);
    SET output = '';
    CASE number % 2
        WHEN 0 THEN
            SET output = CONCAT(number, ' ', 'is an even number');
        ELSE
            SET output = CONCAT(number, ' ', 'is an odd number');
    END CASE;
    SELECT output "Odd Or Even?";
END
```

```
CASE scalar_expression
    WHEN scalar_expression THEN statement_list
    [ WHEN scalar_expression THEN statement_list ]
    ...
    [ ELSE statement_list ]
END CASE;
```

FLOW-CONTROL STATEMENTS:

CASE STATEMENT

```
CREATE PROCEDURE odd_or_even ( IN number INTEGER )
BEGIN
    DECLARE output VARCHAR(255);
    SET output = '';
    CASE
        WHEN number % 2 = 0 THEN
            SET output = CONCAT(number, ' ', 'is an even number');
        ELSE
            SET output = CONCAT(number, ' ', 'is an odd number');
    END CASE;
    SELECT output "Odd Or Even?";
END
```

```
CASE
    WHEN condition THEN statement_list
    [ WHEN condition THEN statement_list ]
    ...
    [ ELSE statement_list ]
END CASE;
```

FLOW-CONTROL STATEMENTS:

WHILE STATEMENT

```
[ label_name : ] WHILE condition DO  
    statement_list  
END WHILE [ label_name ];
```

```
CREATE PROCEDURE show_numbers ( IN upper_limit INTEGER )  
BEGIN  
    DECLARE i INTEGER;  
    CREATE TEMPORARY TABLE sequence_numbers (numbers INTEGER);  
    SET i = 0;  
    WHILE i < upper_limit DO  
        INSERT INTO sequence_numbers(numbers) VALUES(i+1);  
        SET i = i + 1;  
    END WHILE;  
    SELECT numbers "Numbers" FROM sequence_numbers;  
    DROP TEMPORARY TABLE sequence_numbers;  
END
```

FLOW-CONTROL STATEMENTS:

REPEAT STATEMENT

```
CREATE PROCEDURE show_numbers ( IN upper_limit INTEGER )
BEGIN
    DECLARE i INTEGER;
    CREATE TEMPORARY TABLE sequence_numbers (numbers INTEGER);
    SET i = 0;
    REPEAT
        INSERT INTO sequence_numbers(numbers) VALUES(i+1);
        SET i = i + 1;
    UNTIL i >= upper_limit
    END REPEAT;
    SELECT numbers "Numbers" FROM sequence_numbers;
    DROP TEMPORARY TABLE sequence_numbers;
END
```

```
[ label_name : ] REPEAT
    statement_list
UNTIL condition
END REPEAT [ label_name ];
```


FLOW-CONTROL STATEMENTS: LOOP, LEAVE, & ITERATE STATEMENT

```
CREATE PROCEDURE show_odd_numbers ( IN upper_limit INTEGER )
BEGIN
    DECLARE i INTEGER;
    CREATE TEMPORARY TABLE odd_numbers (numbers INTEGER);
    SET i = 0;
    odd: LOOP
        SET i = i + 1;
        IF i > upper_limit THEN LEAVE odd; END IF;
        IF i % 2 = 0 THEN ITERATE odd; END IF;
        INSERT INTO odd_numbers(numbers) VALUES(i);
    END LOOP odd;
    SELECT numbers "Numbers" FROM odd_numbers;
    DROP TEMPORARY TABLE odd_numbers;
END
```

```
[ label_name : ] LOOP
    statement_list
END LOOP [ label_name ];
```

```
LEAVE label_name;
```

```
ITERATE label_name;
```

CALLING STORED PROCEDURES

- A procedure can be called from a program, from interactive SQL, and from stored procedures
- The CALL statement is used

```
CALL [ database_name. ] stored_procedure_name  
    ( [ parameter, parameter, ... ] );
```

- `CALL delete_actor ('A0001');`
- `CALL show_all_movies ();`

QUERYING DATA

- Quite often we want to retrieve data from the tables into the stored procedure
- We can fetch data in two ways
 - If only one-row data must be fetched, we can easily do this with a special version of the SELECT statement: the SELECT INTO statement
 - To fetch multiple rows, the concept of a cursor has been added

QUERYING DATA WITH SELECT INTO

```
SELECT      ...  
INTO        local_variable, local_variable, ...  
FROM        ...  
WHERE       ...  
GROUP BY   ...  
HAVING      ...  
ORDER BY    ...  
LIMIT      ...;
```

- In the INTO clause, we specify the names of variables
- For each expression in the SELECT clause, a variable must be specified
- After processing the SELECT INTO statement, the values of that expression are assigned to the variables

QUERYING DATA WITH SELECT INTO

- `CREATE PROCEDURE count_actors ( OUT counter INTEGER )`
`BEGIN`
 `SELECT COUNT(*)`
 `INTO counter`
 `FROM actors;`
`END`
- `CALL count_actors (@number_of_actors);`
- `SELECT @number_of_actors;`

REMOVING STORED PROCEDURES

- Stored procedures can be removed from the catalog using DROP PROCEDURE statement

```
DROP PROCEDURE [IF EXISTS] [database_name.] procedure_name;
```

- DROP PROCEDURE delete_actor;
- DROP PROCEDURE show_all_movies;

MYSQL DELIMITER

- Each stored program contains a body that consists of an SQL statement
- This statement may be a compound statement made up of several statements separated by semicolon (;) characters
- By default, MySQL itself recognizes the semicolon as a statement delimiter
- You must redefine the delimiter temporarily to cause MySQL to pass the entire stored program definition to the server

MYSQL DELIMITER

- To redefine the MySQL delimiter, use the DELIMITER command
 - `DELIMITER //`
 - ```
CREATE PROCEDURE show_all_movies ()
BEGIN
 SELECT id, name, year, rank
 FROM movies;
END//
```
  - `DELIMITER ;`



# Definition of CTEs

- A **Common Table Expression** (CTE) is like a named subquery. It functions as a virtual table that only its main query can access. CTEs can help simplify, shorten, and organize your code
- CTE provide a way to break down complex queries into more manageable and understandable parts, improving readability and maintainability
- Common Table Expressions (CTEs) are a valuable feature in SQL that provides a way to organize and simplify complex queries. They are particularly useful for recursive queries, data transformation, code reusability, and improving query readability. By mastering CTEs, SQL developers can write more efficient and maintainable code for handling diverse data processing tasks

# Syntax of CTEs

```
WITH cte_name (column1, column2, ...) AS (
 -- CTE query here
 SELECT column1, column2, ...
 FROM table_name
 WHERE conditions
)
-- Main query using the CTE
SELECT *
FROM cte_name;
```

```
WITH my_cte AS (
 SELECT a,b,c
 FROM T1
)
SELECT a,c
FROM my_cte
WHERE
```

# Recursive CTEs

```
WITH RECURSIVE cte_name AS (
 CTE_query_definition -- non recursive query term
 UNION ALL
 CTE_query_definition -- recursive query term
)
SELECT * FROM cte_name;
```

```
WITH RecursiveCTE AS (
 SELECT EmployeeID, Name, ManagerID
 FROM Employees
 WHERE EmployeeID = 1 -- Starting employee ID
 UNION ALL
 SELECT e.EmployeeID, e.Name, e.ManagerID
 FROM Employees e
 JOIN RecursiveCTE r ON e.ManagerID = r.EmployeeID
)
SELECT * FROM RecursiveCTE;
```

- Suppose we have a table Employees with columns EmployeeID, Name and ManagerID where ManagerID refers to the EmployeeID of the manager. We can use a recursive CTE to retrieve the hierarchical structure of employees and their managers
- This recursive CTE starts with the employee whose EmployeeID is 1 and then recursively retrieves all employees reporting to that employee and their managers

# Data Transformation CTEs

```
WITH RevenueByCustomer AS (
 SELECT CustomerID, SUM(TotalAmount) AS TotalRevenue
 FROM Orders
 WHERE YEAR(OrderDate) = 2023
 GROUP BY CustomerID
)
SELECT * FROM RevenueByCustomer;
```

- Suppose we have a table Orders with columns OrderID, CustomerID, OrderDate and TotalAmount. We want to calculate the total revenue generated by each customer in the year 2023
- In this example, the CTE RevenueByCustomer calculates the total revenue for each customer in the year 2023, and then the main query selects and displays the results

## Defines two or more CTEs and use them in main query

```
WITH london1_monthly_revenue AS (
 SELECT
 EXTRACT(MONTH FROM date) as month,
 SUM(unit_price * quantity) AS revenue
 FROM sales
 WHERE EXTRACT(YEAR FROM date) = 2021
 AND branch = 'London-1'
 GROUP BY 1
)
london2_monthly_revenue AS (
 SELECT
 EXTRACT(MONTH FROM date) as month,
 SUM(unit_price * quantity) AS revenue
 FROM sales
 WHERE EXTRACT(YEAR FROM date) = 2021
 AND branch = 'London-2'
 GROUP BY 1
)
SELECT
 l1.month,
 l1.revenue + l2.revenue AS london_revenue,
 l1.revenue AS london1_revenue,
 l2.revenue AS london2_revenue
FROM london1_monthly_revenue l1, london2_monthly_revenue l2
WHERE l1.month = l2.month
```

The query define two CTEs, `london1_monthly_revenue` and `london2_monthly_revenue`, to obtain the monthly revenue in 2021 for each branch in London.

Finally, it joins both CTEs using the month column and calculate the total revenue for London by adding up the revenues of the two branches

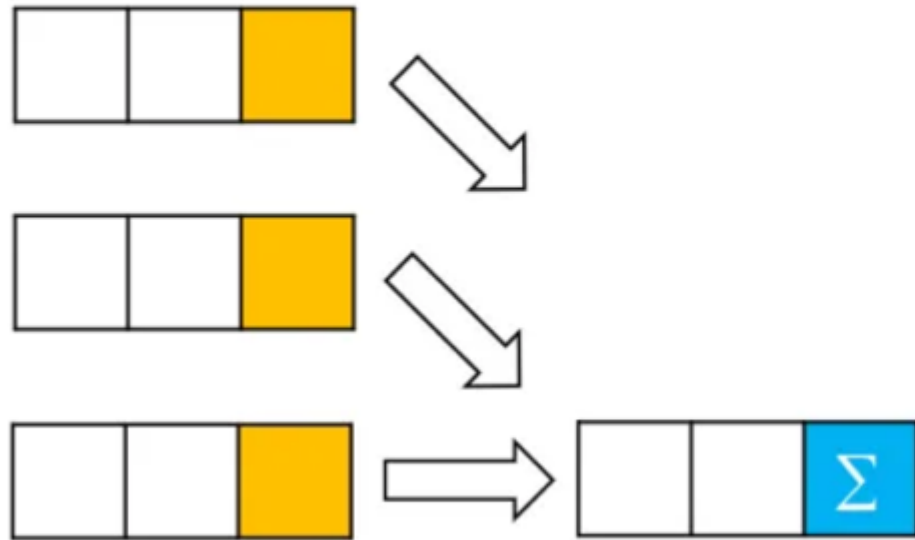
# Definition of SQL Window Function

- A *window function* performs a calculation across a set of table rows that are somehow related to the current row.
- This is comparable to the type of calculation that can be done with an aggregate function. But unlike regular aggregate functions, use of a window function does not cause rows to become grouped into a single output row — the rows retain their separate identities. Behind the scenes, the window function is able to access more than just the current row of the query result.

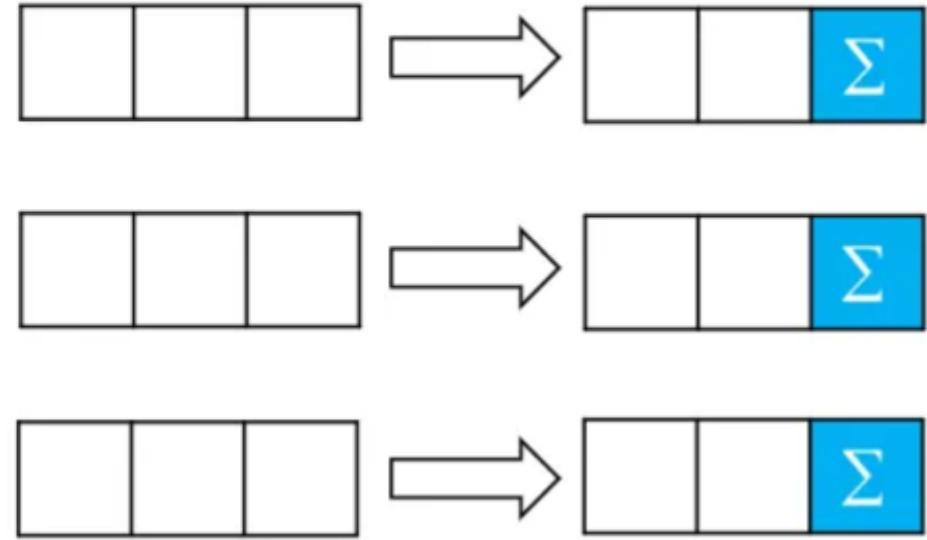
Imagine you're on a sightseeing bus, and you're looking out the window. You see things one after the other, right? SQL Window Functions work a bit like that. They look at your data row by row, but (and here's the cool part) *they remember what they've seen before and what's coming up next*. It's like having a photographic memory while sightseeing!

# Definition of SQL Window Function

## Aggregate Functions



## Window Functions



## Anatomy of a Window Function

```
SELECT column_name,
 WINDOW_FUNCTION(column_name) OVER (
 PARTITION BY column_name
 ORDER BY column_name
 RANGE/ROWS BETWEEN ... AND ...
)
FROM table_name;
```



# Anatomy of a Window Function

- **The Function:** This could be a SUM, AVG, MAX, or any other function you need. It's generally the heart of the (mathematical) operation that you want to perform! These are similar to regular aggregate functions but do not reduce the number of rows returned.
- **OVER():** This part lets SQL know that we're about to do something special, setting the stage for our Window Function. `OVER()` is the cornerstone of window functions in SQL. This clause empowers us to designate a “**window**” or a subset of data that the function will process
- **PARTITION BY:** (Optional) If you want to perform your calculations on specific chunks (groups) of your data, this is how you tell SQL to divide things up. If no `PARTITION BY` is specified, the function treats all rows of the query result set as a single partition. It works similarly to the `GROUP BY` clause, but while `GROUP BY` aggregates the data, `PARTITION BY` doesn't, it just groups the data for the purpose of the window function
- **ORDER BY:** (Optional) This orders the rows within each partition. If no `ORDER BY` is specified, the function treats all rows of the partition as a single group.

# Window Function: Running Total

```
SELECT
 SaleID,
 Salesperson,
 SaleAmount,
 SaleDate,
 SUM(SaleAmount) OVER (ORDER BY SaleDate) AS RunningTotal
FROM Sales;
```



This will calculate the running total of SaleAmount for each row ordered by SaleDate

| SaleID | SaleDate   | Salesperson | SaleAmount | RunningTotal |
|--------|------------|-------------|------------|--------------|
| 1      | 2023-01-01 | Alice       | 300        | 300          |
| 2      | 2023-01-02 | Bob         | 150        | 450          |
| 3      | 2023-01-03 | Alice       | 200        | 650          |
| 4      | 2023-01-04 | Charlie     | 250        | 900          |
| 5      | 2023-01-05 | Bob         | 300        | 1200         |

## Window Function: Cumulative totals (by SalesPerson)

```
SELECT
SaleID,
Salesperson,
SaleAmount,
SaleDate,
SUM(SaleAmount) OVER (PARTITION BY Salesperson ORDER BY
SaleDate) AS CumulativeSalePerPerson
FROM Sales;
```

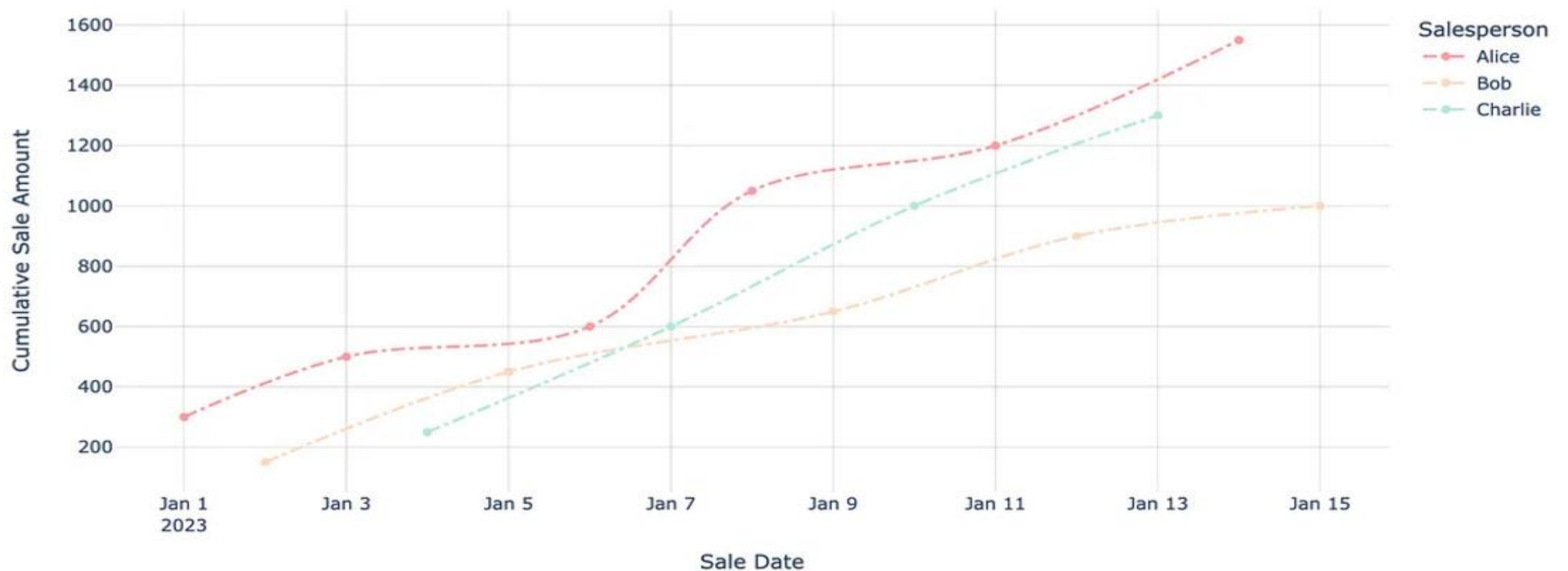


Now what if we wanted to see how each member of the Sales team was evolving over time? It is quite important to keep track of numbers (AKA Quotas) in a Sales team, so we may have a different requirement of actually calculating something like a Running Total not for the whole dataset, but rather for each person on the team

| SaleID | SaleDate   | Salesperson | SaleAmount | CumulativeSalePerPerson |
|--------|------------|-------------|------------|-------------------------|
| 1      | 2023-01-01 | Alice       | 300        | 300                     |
| 2      | 2023-01-02 | Bob         | 150        | 150                     |
| 3      | 2023-01-03 | Alice       | 200        | 500                     |
| 4      | 2023-01-04 | Charlie     | 250        | 250                     |
| 5      | 2023-01-05 | Bob         | 300        | 450                     |

# Window Function: Visualisation of Cumulative totals (by SalesPerson)

Cumulative Sales Performance Over Time



# Window Function: Three days Salesamount

```
SELECT SaleID, SaleDate, Salesperson, SaleAmount,
 AVG(SaleAmount) OVER (
 ORDER BY SaleDate
 ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
) AS MovingAverage
FROM Sales;
```



We used ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING to look at the day before and the day after each row (AKA our “Window”). This is probably the most relevant reason why anyone even thought to call these functions by this name!

| SaleID | Salesperson | SaleAmount | SaleDate   | MovingAverage |
|--------|-------------|------------|------------|---------------|
| 1      | Alice       | 300        | 2023-01-01 | 300.0         |
| 2      | Bob         | 150        | 2023-01-02 | 225.0         |
| 3      | Alice       | 200        | 2023-01-03 | 216.67        |
| 4      | Charlie     | 250        | 2023-01-04 | 200.0         |
| 5      | Bob         | 300        | 2023-01-05 | 250.0         |
| 6      | Alice       | 100        | 2023-01-06 | 216.67        |
| 7      | Charlie     | 350        | 2023-01-07 | 250.0         |
| 8      | Alice       | 450        | 2023-01-08 | 300.0         |
| 9      | Bob         | 200        | 2023-01-09 | 333.33        |
| 10     | Charlie     | 400        | 2023-01-10 | 350.0         |
| 11     | Alice       | 150        | 2023-01-11 | 250.0         |
| 12     | Bob         | 250        | 2023-01-12 | 266.67        |
| 13     | Charlie     | 300        | 2023-01-13 | 233.33        |
| 14     | Alice       | 350        | 2023-01-14 | 300.0         |
| 15     | Bob         | 100        | 2023-01-15 | 250.0         |

# REFERENCES

- Elmasri, Ramez and Shamkant B. Navathe (2016), Fundamentals of Database Systems, 7<sup>th</sup> edition, Addison Wesley.
- Rick F. van der Lans (2007), SQL for MySQL Developers, Addison Wesley.
- Connolly, Thomas and Carolyn Begg (2015), Database Systems A Practical Approach to Design, Implementation, and Management, 6<sup>th</sup> edition, Pearson Education, ISBN 13: 978-1-292-06118-4, Essex, England.
- <https://learnsql.com/blog/what-is-common-table-expression/>
- <https://medium.com/@roshmitadey/common-table-expressions-cte-in-sql-abdf550fc72f>
- <https://medium.com/@manutej/mastering-sql-window-functions-guide-e6dc17eb1995>
- <https://mode.com/sql-tutorial/sql-window-functions>

---

```
SELECT *
FROM WORLD
WHERE SOMEONE
LIKE '%YOU%';
```

```
EMPTY SET (0.00 SEC)
```

---

# Vision

To become an **outstanding** undergraduate Computer Science program that produces **international-minded** graduates who are **competent** in software engineering and have **entrepreneurial spirit** and **noble character**.



# Mission

1. To conduct studies with the best technology and curriculum, supported by professional lecturer
2. To conduct research in Informatics to promote science and technology
3. To deliver science-and-technology-based society services to implement science and technology